PurePy: A Functional Subset of Python for Scientific Computing

TOBIAS ANTENSTEINER, Karlsruhe Institute of Technology, Germany
NIKOLAUS HUBER, University of Cambridge, UK
DOMINIC ORCHARD, University of Kent, UK and University of Cambridge, UK
JACOB PAKE, University of Kent, UK
ROLY PERERA, University of Cambridge, UK and University of Bristol, UK

PurePy is a pure, side-effect-free subset of Python: a functional language whose programs are referentially transparent, reproducible, and easy to reason about, yet remain ordinary Python. Every well-formed PurePy program is valid Python and runs with the same behaviour, so the same source runs under a standard Python interpreter and under any conforming implementation. This paper specifies PurePy as a versioned standard, with a formal grammar, well-formedness rules based on a definite-assignment analysis, and an operational semantics that preserves the meaning of well-formed programs in Python. A reference checker decides membership of the subset, and a conformance test suite is exercised on CPython, PyPy, GraalPy, and Pyodide. Aimed first at researchers in programming languages and pedagogy, PurePy is intended to grow into a common language for scientific computing, supporting portable applications in modelling, data processing, analysis, and visualisation.

1 Introduction

Python is among the most widely used languages in scientific computing, data analysis, and teaching, valued for an accessible surface syntax and a large ecosystem. That reach comes with a large and permissive language. Programs mutate state freely, depend on side effects and dynamic binding, and admit many implicit coercions, so they can be hard to reason about, hard to reproduce, and hard to target with formal tools or with implementations other than the reference interpreter. A researcher in programming languages or in pedagogy who wants a well-behaved functional language faces a trade-off: such a language is easy to reason about but unfamiliar, whereas Python is familiar but unruly.

PurePy carves out a pure, side-effect-free subset of Python that is well-behaved and easy to reason about, yet remains ordinary Python. Every well-formed PurePy program is valid Python and runs under a standard interpreter with the same result, so one artifact serves two roles: a program a scientist can run today, and a common target that a research language or tool can produce and consume. Purity makes programs referentially transparent, deterministic, and reproducible, the properties that scientific computing and formal reasoning both want.

This paper defines PurePy as a versioned formal standard: a formal grammar, well-formedness rules based on a definite-assignment analysis, and an operational semantics that preserves the meaning of well-formed programs in Python. A reference well-formedness checker decides membership of the subset, and a conformance test suite exercises the standard against CPython, PyPy, GraalPy, and Pyodide. The guarantee is precise: membership is decidable, and a well-formed PurePy program is valid Python whose PurePy meaning is its Python meaning.

Authors' Contact Information: Tobias Antensteiner, Institute of Theoretical Informatics, Karlsruhe Institute of Technology, Germany; Nikolaus Huber, Department of Computer Science and Technology, University of Cambridge, UK; Dominic Orchard, School of Computing, University of Kent, UK, Department of Computer Science and Technology and University of Cambridge, UK; Jacob Pake, School of Computing, University of Kent, UK; Roly Perera, Department of Computer Science and Technology, University of Cambridge, UK, School of Computer Science and University of Bristol, UK.

2026. ACM 2475-1421/2026/6-ART1
https://doi.org/10.1145/nnnnnnn.nnnnnnn

The design is deliberately conservative. Rather than reconstruct Python's full behaviour, PurePy chooses a simple, well-behaved core and excludes the idioms that would break it, among them mutable state, effectful control, implicit coercions, and constructs whose meaning depends on module load order (§2). The standard is versioned, so the subset can grow as features earn their place without disturbing programs written against an earlier version.

PurePy is aimed first at researchers in programming languages and in pedagogy, and is intended to grow into a common language for scientific computing, supporting portable applications in modelling, data processing, analysis, and visualisation. Because every PurePy program is also a Python program, adopting it costs nothing at the surface, and a research language that targets PurePy inherits a large and familiar runtime.

1.1 Contributions

After setting out the problem PurePy addresses in §2, we make the following contributions:

- a definition of PurePy as a versioned standard, with a formal grammar and well-formedness rules based on a definite-assignment analysis (§3) and an operational semantics that preserves the meaning of well-formed programs in Python (§4);
- a reference well-formedness checker that decides membership of the subset, validated by a conformance test suite exercised on CPython, PyPy, GraalPy, and Pyodide (§5);
- conservative extensions to the core language, including partial application and piecewise function definitions (§6);
- case studies that use PurePy as a common functional language (§7);
- an experimental evaluation of how well AI agents generate PurePy and translate existing Python into it (§8).

§9 summarises related efforts, and §10 presents our conclusions and discusses future work.

2 Overview

PurePy is a subset of Python chosen so that its programs are well-behaved and broadly *pure*, yet run unchanged under a standard Python interpreter and compute the same result. §2.1 makes the purity claim precise and sets out the excluded features, grouped by the discipline each one protects; Figure 1 summarises the Python runtime errors that well-formedness rules out, and Figure 2 lists planned additions and features under consideration.

2.1 Purity

PurePy is not semantically "pure" in the sense of providing a guarantee that every program terminates with a unique value. No practical programming language is pure in this sense. Real programs may fail to terminate, or abort with errors such as division-by-zero or index-out-of-bounds, even in languages regarded as (mostly) pure, like Haskell.

We do, however, claim that PurePy is pure in the senses that matter. Every name has a unique lexical scope; a variable, once definitely assigned, denotes a fixed value; a module's members are fixed once the module has loaded. There is no mutation, nor can evaluation order change the meaning of a name. Python operates largely outside this discipline, and the goal of PurePy is to characterise a suitably large subset of Python that stays within it.

A non-trivial component of the PurePy specification concerns module loading, in the form of well-formedness rules which rule out programs which depend on Python's load-order-sensitive behaviours (§2.1.4). One might wonder why a language which is mutation-free even needs to concern itself with loading order. If the result of a program is all that can be observed, the order in which subexpressions are evaluated cannot affect that result, and so cannot meaningfully be

part of the specification. However runtime errors, at least if they can be distinguished, remove that freedom: if division-by-zero reports differently from index-out-of-bounds, a program with two possible failures reveals which was reached first, and implementations which choose different orders might disagree about what such a program does. PurePy aims to serve as a template for practical implementation, not as an idealised calculus, and practical implementations need to be able to report on different kinds of error, since those reports are important to users. The sequential semantics of module loading therefore remains an important part of the specification so that programs which fail do so deterministically under every implementation.

Real-world languages, including the "pure" ones, also support logging to a terminal. Haskell has `trace`; PurePy has `print`, following Python. This too is a computational effect, but one that is not only indispensable but benign; printing can only report on a computation and cannot influence its result. Moreover with loading order already part of the specification, printing is a low-cost, high-utility feature that leaves the core purity benefits of PurePy untouched.

2.1.1 Mutable state. A variable in PurePy denotes a fixed value once definitely assigned, so features that mutate or rebind state are excluded: `global` and `nonlocal`, which reassign bindings in an outer scope; `del`, which removes a binding; augmented assignment (`+=`,`-=` , and so on); and the walrus operator (`:=`), which assigns within an expression. Reassignment of a variable already captured by a closure is excluded for the same reason. In Python a closure captures a mutable binding, so a later reassignment is visible through the closure:

```
def f():
    x = 5
    def g():
        return x
    x = 6
    return x + g()
```

Here `g()` returns `6` in Python and the result is `12`; under a pure reading in which `x = 6` shadows the earlier binding, `g()` would return `5` and the result `11`. PurePy rejects the program, keeping closures over values rather than mutable cells, as with Java's requirement that captured locals be effectively final [Gosling et al. 2025, §15.27.2]. The rules that enforce this are given in §3.4.4.

2.1.2 Control flow and effects. PurePy excludes loops with `break` and `continue`, exceptions (`try`/`except`/`raise`), context managers (`with`), generators (`yield`/`yield from`), and coroutines (`async`/`await`). Functional programming has rich accounts of effects, from monads to algebraic effect handlers, so these features are not inherently at odds with a functional language. In idiomatic Python, though, they are used mainly in conjunction with mutable state, which PurePy already excludes, and iteration is expressed by comprehensions and recursion rather than by loops. A future release may add structured effects; for now the core stays effect-free.

2.1.3 Implicit coercions. Python accepts several implicit coercions and lax matches that are convenient but a recognised source of error, since what Python accepts rarely coincides with what the author meant. PurePy excludes three. The first is truthiness: the test of an `if` or conditional expression, or an `assert`, is coerced to `bool`, whereas PurePy requires it to be boolean already. The second is between sequence kinds: Python lets a list pattern match a tuple value, and a tuple pattern match a list, whereas in PurePy a list pattern matches only a list and a tuple pattern only a tuple. Subscripting draws the same line, applying to lists alone, a tuple being taken apart by pattern matching. The third is in constructor patterns: Python lets a positional class pattern supply fewer sub-patterns than the class's field arity, matching only those given, whereas PurePy requires the

exact arity. Only the third is decidable without types, so the first two are stated here rather than enforced by the well-formedness rules.

2.1.4 Load-dependent meaning. A name's meaning in PurePy must not depend on the order in which modules load or on runtime state, so several of Python's import and module behaviours are excluded. Imports must appear at the start of a module, never inside a function or other block, nor after a non-import statement. Wildcard imports (`from q import *`) are excluded, since the names they introduce are not explicit; PEP 8 recommends against them.[1] A submodule may not import itself through its parent (`from a import b` inside module `a.b`): PurePy treats the from-imported submodule as a dependency, so this is an import cycle. An `import` (as opposed to a from-import) may not name a proper descendant of the importing module (`import x.y` inside module `x`), since the binding for the head would denote the still-loading module.

A module may not bind a name that is also the name of one of its submodules. In Python an attribute of a package can be set both by an assignment in the package body and by loading the same-named submodule, and which wins depends on load order. Consider a program with four modules:

```
# pkg              # pkg.sub          # helper               # __main__
sub = 1            # (empty)          import pkg.sub         import pkg
                                                             print(pkg.sub)  # 1
```

Here `helper` is never imported, so `pkg.sub` never loads, and `pkg.sub` in the main module denotes the assignment. Now let the main module also import `helper`, which it does not otherwise use:

```
# __main__
import helper
import pkg
print(pkg.sub)  # <module 'pkg.sub'>
```

Loading `pkg.sub`, wherever in the program it happens, sets it as an attribute of the shared `pkg` object, so the extra import changes what `pkg.sub` means in a module that never mentions `pkg.sub`. The name has no load-order-independent meaning, so PurePy prohibits the clash outright.

For the same reason, a module may not refer to a submodule it does not itself import. A submodule is not an attribute of its package until it is loaded, so `import a` followed by `a.b.f()` is rejected unless the module also imports `a.b`; Python permits the reference whenever some earlier import, anywhere in the program, happens to have loaded `a.b`. PurePy likewise rejects a reference to a module member the module body never assigns, such as `lib.x` where `lib` assigns `x` only on a branch that is not taken. In Python both cases raise `AttributeError`; PurePy rejects them statically.

2.1.5 Modules and classes are not first-class. For now, modules and classes are not values in PurePy. In Python a module is a value and a class declaration is an assignment, so both flow through programs as data. Treating them as first-class would complicate the static account, so the present subset keeps them second-class rather than ruling the feature out; a later release may revisit it. A module name may be projected by member access but not used as a value, and a class name appears only at constructor and pattern positions, with declarations confined to module level. Because a class is identified by its qualified name, a dataclass declaration may not re-declare a name that already denotes a class in the module; Python permits the re-declaration, leaving the earlier class reachable only through its existing instances.

[1] https://peps.python.org/pep-0008/

Python error	Ruled out when	See
`NameError`, `UnboundLocalError`	Always: a variable may be referenced only where it is definitely assigned.	Figure 8, Figure 9
`ModuleNotFoundError`	Always: an import requires its target to be a module of the program.	Figure 11
`ImportError`	Always: a from-import requires each imported name to be a member of the source module, and import cycles are rejected. (A Python cycle may run or fail, depending on where the cycle is entered.)	Figure 10, Figure 12
`AttributeError`	Always: a module attribute reference must resolve to a definitely assigned member.	Figure 9
`TypeError`	The operation is a constructor call or class pattern, which must match the class's arity exactly. (Only excess sub-patterns are a Python error; a pattern with fewer is prohibited even though Python permits it.)	Figure 9, Figure 7

Fig. 1. Python errors prevented by well-formedness

To do		**Under consideration**	
Reserved words	#9	Destructuring assignment	#54
Set and dict comprehensions	#52	f-strings	#55
Set and dict literals	#87	Default arguments	#56
Membership operator (`in`)	#80	`*args` and `**kwargs`	#57
Identity operator (`is`)	#81	Decorators	#58
Chained comparison	#82	Slicing	#59
Enums	#86	Star patterns	#84
Remaining expression rules	#133	Or patterns	#85
Aliased imports	#135	Relative imports	#126
		Wildcard imports	#105
		Function keyword arguments	#127
		Postponed annotation evaluation	#128
		Primitive operations	#132

Fig. 2. [**TODO:** Planned additions and features under consideration]

2.1.6 Other restrictions. A few remaining restrictions do not fit the groups above. Only dataclasses are supported, so `class` definitions of other kinds are excluded.

Complementing the exclusions above, a well-formed PurePy program avoids the Python runtime errors listed in Figure 1, under the conditions noted there. Errors from ill-typed operations, such as wrong operand types, calling a non-function, or attribute access on an object lacking the attribute, are outside the scope of well-formedness.

3 Syntax

The syntax of PurePy is given in Figure 3 and Figure 4. A *block* is a non-empty sequence of statements; either a top-level statement, a `def` body, or a branch of a conditional. A *mutual region* $d_1 \ldots d_{n+1}$ is a maximal contiguous non-empty sequence of `def` statements in a block: any non-`def` statement in that block, by definition, separates mutual regions. Maximality is not enforced by the grammar in Figure 3; the parser is responsible for grouping consecutive `def`s into a single region. The well-formedness rules treat each region as one mutually-recursive binding (see §3.4.5).

3.1 Contexts

Metavariables x, f, c range over *variables*; idiomatically f is used when the variable is expected to denote a function and c when it is expected to denote a class.

Definition 1 (Context). *A context Γ (or Δ) is a finite map from variables to one of the following* context entries θ*:*

- *the variable's* definite assignment *status* $a \in \mathbb{B} = \{\mathsf{tt}, \mathsf{ff}\}$*;*
- *a* module reference*, for a fully-qualified module name q: either a* stub $\mathrm{Mod}(q)$*, or* loaded *reference* $\mathrm{Mod}(q, \Gamma)$ *carrying the* signature Γ *of q, the context specifying its members;*
- *a* class entry C*, of the form* $\mathrm{Cls}_\Gamma(q.c, \vec{x}, c')$*, recording the declaring context Γ, the qualified name $q.c$ of the class, sequence of distinct field names $\vec{x}$, and base class name c' (or $\bot$ for no base).*

Module references arise from import statements. A module declares each class name at most once, so the qualified name of a class entry is the identity of the class. The definite-assignment terminology is borrowed from Java [Gosling et al. 2025, §16]; tt indicates *definitely assigned* and ff *not definitely assigned* (or, equivalently, degenerate types "any" and "none"). It is useful to represent contexts as total functions on the set of all variables by extending the codomain with $\bot$ to indicate *undefined*. We write $\Gamma, x{:}\,\theta$ for the context that sends x to θ and otherwise agrees with Γ, define $\mathrm{dom}(\Gamma) = \{x \mid \Gamma(x) \neq \bot\}$, and write $\varnothing$ for the context where all variables are undefined. For a set of variables X, write $\mathsf{tt}(X)$ for the context $\{x{:}\,\mathsf{tt} \mid x \in X\}$; similarly $\mathsf{ff}(X)$.

Context operations. Contexts admit a non-commutative monoid of sequential composition $\Gamma \cdot \Delta$ ("override") and an idempotent commutative semigroup of parallel composition $\oplus$ ("merge"). Together these form a *right near-semiring* [van Hoorn and van Rootselaar 1967]: $\cdot$ right-distributes over $\oplus$ (Lemma 1), but left distributivity does not hold. Both operations are defined pointwise.

Definition 2 (Context override).

$$\begin{aligned} \bot \cdot \theta = \theta \cdot \bot = \theta \\ \theta \cdot \theta' = \theta' \quad (\textit{otherwise}) \end{aligned}$$

Override is thus right-biased and has $\bot$ as both left and right unit.

Definition 3 (Context merge). *For definite-assignment statuses $a, a' \in \mathbb{B}$:*

$$\begin{aligned} \bot \oplus \bot = \bot \\ a \oplus \bot = \bot \oplus a = \mathsf{ff} \\ a \oplus a' = a \wedge a' \end{aligned}$$

Merge is otherwise undefined. Restricted to $\mathbb{B}$, it returns ff when exactly one argument is defined and the conjunction otherwise; so $\bot$ is not a unit.

Definition 4 (Context extend).

$$\begin{aligned} \mathrm{Mod}(q, \Gamma) \lhd \mathrm{Mod}(q, \Gamma') = \mathrm{Mod}(q, \Gamma \lhd \Gamma') \\ \mathrm{Mod}(q) \lhd \mathrm{Mod}(q, \Gamma) = \mathrm{Mod}(q, \Gamma) \lhd \mathrm{Mod}(q) = \mathrm{Mod}(q, \Gamma) \\ \bot \lhd \theta = \theta \lhd \bot = \theta \\ \theta \lhd \theta' = \theta' \quad (\textit{otherwise}) \end{aligned}$$

Extend combines module references for the same module recursively, preferring loaded references to stubs, and is otherwise right-biased like override. It is used when processing import prefixes, where two imports rooted at the same package must combine rather than shadow.

$$
\begin{array}{rcll}
s & ::= & \texttt{pass} & \text{(Pass)} \\
& | & x = e & \text{(Assign)} \\
& | & e & \text{(ExprStmt)} \\
& | & \texttt{assert } e & \text{(Assert)} \\
& | & \texttt{assert } e, e' & \text{(AssertMsg)} \\
& | & \texttt{return} & \text{(ReturnNone)} \\
& | & \texttt{return } e & \text{(Return)} \\
& | & \texttt{if } e : \ b \ \texttt{elif } e_1 : \ b_1 \ \ldots \ \texttt{elif } e_n : \ b_n & \text{(If)} \\
& | & \texttt{if } e : \ b \ \texttt{elif } e_1 : \ b_1 \ \ldots \ \texttt{elif } e_n : \ b_n \ \texttt{else} : \ b' & \text{(IfElse)} \\
& | & \texttt{match } e\texttt{: case } p_1\texttt{: } b_1 \ \ldots \ \texttt{case } p_{n+1}\texttt{: } b_{n+1} & \text{(Match)} \\
& | & d_1 \ \ldots \ d_{n+1} & \text{(MutualRegion)} \\
& | & \texttt{@dataclass class } c : \ \phi & \text{(Dataclass)} \\
& | & \texttt{@dataclass class } c(c') : \ \phi & \text{(DataclassExtend)} \\
\\
\iota & ::= & \texttt{import } q & \text{(Import)} \\
& | & \texttt{from } q \texttt{ import } x_1, \ldots, x_{n+1} & \text{(FromImport)} \\
\\
\phi & ::= & x_1 \texttt{ : Any } \ldots \ x_{n+1} \texttt{ : Any} & \text{(FieldList)} \\
& | & \texttt{pass} & \text{(EmptyFieldList)} \\
\\
b & ::= & s_1 \ \ldots \ s_{n+1} & \text{(Block)} \\
\\
t & ::= & \iota_1 \ \ldots \ \iota_m \ \ s_1 \ \ldots \ s_n & \text{(ModuleBody)} \\
\\
q & ::= & x_1.x_2.\cdots.x_{n+1} & \text{(Name)} \\
\\
d & ::= & \texttt{def } f(x_1, \ldots, x_n) : \ b & \text{(Def)}
\end{array}
$$

Fig. 3. Modules and blocks

Lemma 1 (Right distributivity).

$$(\Delta \oplus \Delta') \cdot \Gamma = (\Delta \cdot \Gamma) \oplus (\Delta' \cdot \Gamma).$$

Left distributivity ($\Gamma \cdot (\Delta \oplus \Delta') = (\Gamma \cdot \Delta) \oplus (\Gamma \cdot \Delta')$) does not hold. If it did, the following two programs would be equivalent, since sequential composition with a preceding assignment could be distributed into the branches of a conditional. But they are not: the left program is ill-formed, because a variable assigned before a conditional does not remain definitely assigned if only some branches assign it (see also §3.4.2).

```
def f(b):
    x = 1
    if b:
        x = 2
    else:
        pass
    return x  # error
```

```
def f(b):
    if b:
        x = 1
        x = 2
    else:
        x = 1
        pass
    return x  # ok
```

Operators. The metavariable $\odot$ ranges over the supported binary operators other than `and` and `or`, and $\ominus$ over the supported unary operators, both enumerated in Figure 5. The operators `and` and `or` have their own expression forms; they do not evaluate both operands. The grammar of

e	::=	x	(Var)
	|	ℓ	(Literal)
	|	$e(e_1, \ldots, e_n)$	(Call)
	|	$q(e_1, \ldots, e_n, x_{n+1}{=}e_{n+1}, \ldots, x_{n+m}{=}e_{n+m})$	(Constr)
	|	$\{x_1{:}e_1, \ldots, x_n{:}e_n\}$	(Dict)
	|	$e.x$	(Attribute)
	|	$e[e']$	(Subscript)
	|	$e \odot e'$	(BinOp)
	|	$\ominus e$	(UnaryOp)
	|	e `and` e'	(And)
	|	e `or` e'	(Or)
	|	e_1 `if` e `else` e_2	(CondExpr)
	|	$[e_1, \ldots, e_n]$	(List)
	|	$(e_1, \ldots, e_n)$	(Tuple)
	|	[e g_1 ... g_n]	(ListComp)
	|	`lambda` $x_1, \ldots, x_n$: e	(Lambda)
g	::=	`if` e	(Guard)
	|	`for` x `in` e	(Generator)
ℓ	::=	n	(Number)
	|	w	(String)
	|	`True`	(True)
	|	`False`	(False)
	|	`None`	(None)
p	::=	ℓ	(PatLiteral)
	|	$-n$	(PatNegLit)
	|	x	(PatVar)
	|	_	(PatWildcard)
	|	$[p_1, \ldots, p_n]$	(PatList)
	|	$(p_1, \ldots, p_n)$	(PatTuple)
	|	$q(p_1, \ldots, p_n, x_{n+1}{=}p_{n+1}, \ldots, x_{n+m}{=}p_{n+m})$	(PatConstr)
	|	p `as` x	(PatAs)

Fig. 4. Expressions and patterns

Level	Operators	Arity	Associativity
1 (lowest)	`or`	binary	left
2	`and`	binary	left
3	`not`	unary	—
4	`==` `!=` `<` `<=` `>` `>=`	binary	left
5	`+` `-`	binary	left
6	`*` `/` `//` `%`	binary	left
7	`+` `-`	unary	—
8 (highest)	`**`	binary	right

Fig. 5. Operator precedence (lowest binds loosest); follows Python

Figure 4 is intentionally ambiguous on operator forms; disambiguation is by the precedence and associativity given in Figure 5, which follow Python.

Module	Members
builtins	print, len, range
math	pi, e, sqrt, exp, log, sin, cos, tan, floor, ceil
sys	argv, exit
typing	Any
dataclasses	dataclass

Fig. 6. Predefined modules and their members

3.2 Modules and programs

A PurePy *program* is a finite map $\mathcal{M}$ from *fully-qualified module names* (Figure 3) to *module bodies* t, of which one is a distinguished *main module* named `__main__`; here name(q) is the string literal naming q, so name(`__main__`) = `"__main__"`. We identify a module with its fully-qualified name, writing "module q" for the module named q. A module body is a sequence of imports $\vec{\imath}$ followed by a sequence of statements $\vec{s}$; since imports are not statements, they can appear neither in blocks (and hence not in function bodies) nor after the first non-import statement. The well-formedness and evaluation rules check and evaluate the statements of each body $\mathcal{M}(q)$ with the assignment `__name__` = name(q) prepended, modelling the `__name__` binding the runtime supplies automatically. $\mathcal{M}$ additionally always includes the predefined modules of Figure 6, which have no body and whose contexts and environments are given directly (Definition 5), and is prefix-closed: the parent of any module in dom($\mathcal{M}$) is itself in dom($\mathcal{M}$).[2] The map $\mathcal{M}$ is fixed and ambient in all definitions that follow, as is the name q of the enclosing module in the rules that check or evaluate a single module's body.

Definition 5 (Predefined contexts and environments). *For each predefined module q of Figure 6, the context Γ_q binds the members listed for q, together with `__name__`, all definitely assigned. The dynamic semantics is parameterised by an environment ρ_q for each such q, binding the same names to implementation-defined values.*

The metafunction assigns extends by union from individual statements to blocks; on a module body $\vec{\imath}\ \vec{s}$ it acts on $\vec{s}$. Write $q \preceq q'$ (*prefix*) when $q' = q$ or $q' = q.q''$ for some q'', and $\prec$ for the strict order; the proper prefixes of a module name are its *ancestors*. The submods metafunction is defined, with the class-related metafunctions, in the appendix of the full version.

In this section, loading a module means checking its body and computing its signature (Figure 12), the static counterpart of run-time loading (§4). Both import forms load their target, together with its ancestors (Figure 11). A plain import also records each of these modules as a member of its parent (the loads-as judgement), so that the binding created for the root of the target gives attribute access along the whole chain. A from-import also loads any imported name that denotes a submodule. An ancestor of the target is exempt from loading first if it is the importing module or one of its ancestors, since at run time it may still be loading. The exemption applies only to from-imports, because for a plain import the only ancestor that could never load first is the importing module itself, and an import naming a proper descendant of the importing module is prohibited outright (§2.1.4). Loading a module may thus require other modules to load first; the rules are interpreted inductively, so a program with an import cycle is ill-formed, because a module in a cycle could finish loading only after itself. Program well-formedness (Figure 12) requires the main module to

[2] A source tree encodes such an $\mathcal{M}$: each module's body is its `.py` file, each package's body is its `__init__.py`, and a package directory without an `__init__.py` encodes the empty body (a namespace package, PEP 420, https://peps.python.org/pep-0420/). Prefix-closure thus holds by construction.

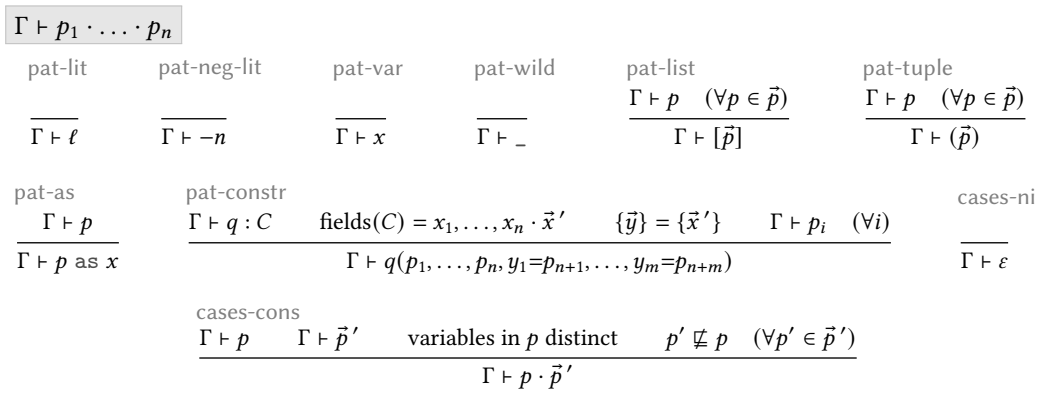


Fig. 7. Case patterns are well-formed under Γ

be well-formed and hence, recursively, every module the program can load; a module that is never imported is not checked.

3.3 Well-formedness

3.3.1 Auxiliary definitions. Three syntactic analyses support the well-formedness rules: assigns(s), the set of variables assigned anywhere in s (without descending into nested function definitions, which have their own scope, so that assigns(s) = dom(Δ) when s : Assigns Δ); binds(p), the set of variables introduced by a pattern p; and captures(s), the set of variables from the enclosing scope that are captured by closures (lambdas or nested function definitions) within s, with companion $\text{captures}_\text{e}(e)$ on expressions. All are defined, by their principal cases, in the appendix of the full version. captures extends to blocks and module bodies by union (like assigns), with the singleton block identified with its statement.

Definition 6 (Pattern subsumption). *The preorder $p \sqsubseteq p'$ (the appendix of the full version) holds when every value matched by p is also matched by p'.*

Subsumption is used by the well-formedness rule for `match` to rule out unreachable cases and to check exhaustiveness; with the current pattern set, the latter reduces to requiring a catch-all (var or wildcard) case.

3.3.2 Well-formedness rules. A pattern list is well-formed (Figure 7) when each pattern is linear (no repeated variable names) and no earlier pattern subsumes a later one. The `match` well-formedness rules use this judgement to enforce reachability and linearity of case patterns.

Every well-formed statement either *definitely returns*, or *definitely assigns* a context Δ, which may be empty. The base cases are that `return e` definitely returns, and `x = e` definitely assigns $\{\texttt{x}\colon \texttt{tt}\}$. *Result types* R classify these outcomes: R is either Returns (definitely returns) or Assigns Δ (definitely assigns Δ). Block well-formedness is given in Figure 8; expression well-formedness in Figure 9.

Context operations $\cdot$ and $\oplus$ lift to a right semiring over result types R, with Assigns $\varnothing$ as left and right unit for $\cdot$ and Returns as unit for $\oplus$ and zero for $\cdot$.

Definition 7 (Override and merge on result types).

$$\text{Assigns } \Delta \oplus \text{Assigns } \Delta' = \text{Assigns } (\Delta \oplus \Delta')$$
$$R \oplus \text{Returns} = \text{Returns} \oplus R = R$$

$$\text{Assigns } \Delta \cdot \text{Assigns } \Delta' = \text{Assigns } (\Delta \cdot \Delta')$$
$$R \cdot \text{Returns} = \text{Returns} \cdot R = \text{Returns}$$

The $\text{Returns} \cdot R$ case never arises because the well-formedness judgement prohibits unreachable statements (§3.4.7).

Lemma 2 (Right distributivity (result types)).

$$(R \oplus R') \cdot S = (R \cdot S) \oplus (R' \cdot S)$$

A module attribute reference is well-formed only when it resolves to a definitely assigned member (rule attr-module); see §2.1.4.

The signature Γ a module's load judgement produces (Figure 12) comprises the classes and variables a module defines, together with its intrinsic `__name__` and a stub for each submodule, never the names it imports: following PEP 484[3], an imported name is a private dependency, not re-exported as part of the interface. Submodule stubs and the module's own bindings are disjoint, because the module rules prohibit a module from binding a name that is also the name of a submodule (§2.1.4).

3.4 Examples

3.4.1 Assignment as definition. Although PurePy is pure and all variable usage is well-scoped, local variables are introduced using an *assignment statement*, for compatibility with Python. If a variable is definitely assigned in any branch of a conditional, then for that variable to be usable after the conditional, every other branch of the conditional must either definitely return, or definitely assign that variable too. Thus the following are both well-formed:
The definite assignment set for a conditional that has at least one branch which definitely assigns is calculated by merging the definite assignment sets for all branches which definitely assign, using the $\oplus$ operator on environments.

Definite assignment thus gives each reference a unique lexical referent: a variable is introduced by an assignment, possibly jointly by the branches of a conditional, and a name may be referenced only where one such introduction covers every path. This is stricter than avoiding Python's `NameError`: the left-hand program of §3.1 assigns `x` on every run, but is rejected because the reference would denote the branch-local variable on one path and the variable it hides on the other.

3.4.2 Local assignment in conditional clauses. For flexibility, we permit *branch-local* variables. The assignment of `x` below (left) is permitted, and can be used locally in that branch of the conditional: However such variables only appear to be branch-local. The definition of $\oplus$ actually ensures that a variable assigned in any branch is considered assigned by the conditional; however unless it is assigned in *every* branch that does not definitely return, it will be bound to ff in the corresponding environment. Any attempt to refer to such a variable is illegal, as it refers to a variable which may not be assigned. (Thus the right-hand program is ill-formed, despite the enclosing `x = 6`, because in the `return x` statement, the inner assignment shadows the the outer one, and the inner assignment is only partial.) If we were to allow the program on the right, there would be no straightforward way to assign it a semantics which is both pure, and in agreement with Python, since in Python the function returns `3` in the case where `b` is `True`.

[3] https://peps.python.org/pep-0484/

$\Gamma \vdash b : R$

pass
$$\frac{}{\Gamma \vdash \texttt{pass} : \text{Assigns } \varnothing}$$

assign
$$\frac{\Gamma \vdash e \qquad x \notin \text{captures}_e(e)}{\Gamma \vdash x = e : \text{Assigns } \{x : \text{tt}\}}$$

expr-stmt
$$\frac{\Gamma \vdash e}{\Gamma \vdash e : \text{Assigns } \varnothing}$$

assert
$$\frac{\Gamma \vdash e}{\Gamma \vdash \texttt{assert } e : \text{Assigns } \varnothing}$$

assert-msg
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash \texttt{assert } e, e' : \text{Assigns } \varnothing}$$

return-none
$$\frac{}{\Gamma \vdash \texttt{return} : \text{Returns}}$$

return
$$\frac{\Gamma \vdash e}{\Gamma \vdash \texttt{return } e : \text{Returns}}$$

if
$$\frac{\Gamma \vdash e_i \quad (\forall i) \qquad \Gamma \vdash b_i : R_i \quad (\forall i)}{\Gamma \vdash \texttt{if } e_1 : \ b_1 \ \texttt{elif } e_2 : \ b_2 \ \ldots \ \texttt{elif } e_{n+1} : \ b_{n+1} : R_1 \oplus \ldots R_{n+1} \oplus \ldots \oplus \text{Assigns } \varnothing}$$

if-else
$$\frac{\Gamma \vdash e_i \quad (\forall i) \qquad \Gamma \vdash b_i : R_i \quad (\forall i)}{\Gamma \vdash \texttt{if } e_1 : \ b_1 \ \texttt{elif } e_2 : \ b_2 \ \ldots \ \texttt{elif } e_{n+1} : \ b_{n+1} \ \texttt{else} : \ b_{n+2} : R_1 \oplus \ldots \oplus R_{n+2}}$$

def
$$\frac{\Gamma \cdot \text{tt}(\vec{f} \cdot \vec{x}_i) \cdot \text{ff}(\text{assigns}(b_i) \setminus \vec{x}_i) \vdash b_i : R_i \quad (\forall i) \qquad \vec{f} \text{ distinct}}{\Gamma \vdash \texttt{def } f_1(\vec{x}_1) : \ b_1 \cdot \ldots \cdot \texttt{def } f_{n+1}(\vec{x}_{n+1}) : \ b_{n+1} : \text{Assigns } \text{tt}(\vec{f})}$$

match
$$\frac{\Gamma \vdash e \qquad \Gamma \cdot \text{tt}(\text{binds}(p_i)) \vdash b_i : R_i \quad (\forall i) \qquad S_i = \text{Assigns } \text{tt}(\text{binds}(p_i)) \cdot R_i \quad (\forall i) \qquad \Gamma \vdash \vec{p} \qquad p_{n+1} \in \{x, _\}}{\Gamma \vdash \texttt{match } e\texttt{: case } p_1\texttt{: } b_1 \ \ldots \ \texttt{case } p_{n+1}\texttt{: } b_{n+1} : S_1 \oplus \ldots \oplus S_{n+1}}$$

match-partial
$$\frac{\Gamma \vdash e \qquad \Gamma \cdot \text{tt}(\text{binds}(p_i)) \vdash b_i : R_i \quad (\forall i) \qquad S_i = \text{Assigns } \text{tt}(\text{binds}(p_i)) \cdot R_i \quad (\forall i) \qquad \Gamma \vdash \vec{p} \qquad p_{n+1} \notin \{x, _\}}{\Gamma \vdash \texttt{match } e\texttt{: case } p_1\texttt{: } b_1 \ \ldots \ \texttt{case } p_{n+1}\texttt{: } b_{n+1} : S_1 \oplus \ldots \oplus S_{n+1} \oplus \text{Assigns } \varnothing}$$

cons
$$\frac{\Gamma \vdash s : \text{Assigns } \Delta \qquad \Gamma \cdot \Delta \vdash b : R \qquad \text{captures}(s) \cap \text{assigns}(b) = \varnothing}{\Gamma \vdash s \cdot b : \text{Assigns } \Delta \cdot R}$$

Fig. 8. Block b is well-formed in Γ

3.4.3 Function-level scoping of local variables. In Python, all local variables are scoped to the entire function body, not just from the point of assignment onwards. The assigns function in the mutual rule reflects this: all variables assigned anywhere in the function body are introduced at ff before the body is checked. This means the following program is statically rejected by PurePy, whereas Python rejects it at runtime with an `UnboundLocalError`:

```
y = 7

def f():
    x = y  # error: y is local (due to assignment below)
    y = 8
```

$\Gamma \vdash q : \theta$

simple-var
$$\frac{x{:}\,\mathrm{tt} \in \Gamma}{\Gamma \vdash x : \mathrm{tt}}$$

simple-module
$$\frac{x{:}\,\mathrm{Mod}(q, \Gamma') \in \Gamma}{\Gamma \vdash x : \mathrm{Mod}(q, \Gamma')}$$

simple-class
$$\frac{c{:}\,C \in \Gamma}{\Gamma \vdash c : C}$$

qualified
$$\frac{\Gamma \vdash q : \mathrm{Mod}(q', \Gamma') \qquad \Gamma' \vdash x : \theta}{\Gamma \vdash q.x : \theta}$$

(a) Name q resolves to context entry θ under Γ

$\Gamma \vdash e$

var
$$\frac{x{:}\,\mathrm{tt} \in \Gamma}{\Gamma \vdash x}$$

var-builtin
$$\frac{x \notin \mathrm{dom}(\Gamma) \qquad x{:}\,\mathrm{tt} \in \Gamma_{\texttt{builtins}}}{\Gamma \vdash x}$$

literal
$$\frac{}{\Gamma \vdash \ell}$$

lambda
$$\frac{\Gamma \cdot \mathrm{tt}(\vec{x}) \vdash e}{\Gamma \vdash \texttt{lambda}\ \vec{x}\texttt{:}\ e}$$

call
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e_i \quad (\forall i)}{\Gamma \vdash e(e_1, \ldots, e_n)}$$

constr
$$\frac{\Gamma \vdash q : C \qquad \mathrm{fields}(C) = x_1, \ldots, x_n \cdot \vec{x}\,' \qquad \{\vec{y}\} = \{\vec{x}\,'\} \qquad \Gamma \vdash e_i \quad (\forall i)}{\Gamma \vdash q(e_1, \ldots, e_n, y_1{=}e_{n+1}, \ldots, y_m{=}e_{n+m})}$$

attr-module
$$\frac{\Gamma \vdash e.x : \mathrm{tt}}{\Gamma \vdash e.x}$$

attr-object
$$\frac{\Gamma \vdash e}{\Gamma \vdash e.x}$$

list
$$\frac{\Gamma \vdash e_i \quad (\forall i)}{\Gamma \vdash [e_1, \ldots, e_n]}$$

tuple
$$\frac{\Gamma \vdash e_i \quad (\forall i)}{\Gamma \vdash (e_1, \ldots, e_n)}$$

subscript
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash e[e']}$$

binop
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash e \odot e'}$$

unary-op
$$\frac{\Gamma \vdash e}{\Gamma \vdash \ominus e}$$

and
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash e\ \texttt{and}\ e'}$$

or
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e'}{\Gamma \vdash e\ \texttt{or}\ e'}$$

cond
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash e_1 \qquad \Gamma \vdash e_2}{\Gamma \vdash e_1\ \texttt{if}\ e\ \texttt{else}\ e_2}$$

list-comp
$$\frac{\Gamma \vdash \vec{g} : \Delta \qquad \Gamma \cdot \Delta \vdash e}{\Gamma \vdash [\, e\ g_1\ \ldots\ g_n]}$$

(b) Expression e is well-formed under Γ

$\Gamma \vdash \vec{g} : \Delta$

quals-nil
$$\frac{}{\Gamma \vdash \varepsilon : \varnothing}$$

qual-guard
$$\frac{\Gamma \vdash e \qquad \Gamma \vdash \vec{g} : \Delta}{\Gamma \vdash \texttt{if}\ e \cdot \vec{g} : \Delta}$$

qual-generator
$$\frac{\Gamma \vdash e \qquad \Gamma \cdot \{x{:}\,\mathrm{tt}\} \vdash \vec{g} : \Delta}{\Gamma \vdash \texttt{for}\ x\ \texttt{in}\ e \cdot \vec{g} : \{x{:}\,\mathrm{tt}\} \cdot \Delta}$$

(c) Comprehension qualifiers $\vec{g}$ bind Δ under Γ

Fig. 9. Expression well-formedness

Γ, x imports θ

imp-member
$$\frac{\Gamma \vdash x : \theta}{\Gamma, x \text{ imports } \theta}$$

imp-submodule
$$\frac{x{:}\,\mathrm{Mod}(q) \in \Gamma \qquad \vdash_{\mathsf{load}} q : \Delta}{\Gamma, x \text{ imports } \mathrm{Mod}(q, \Delta)}$$

Fig. 10. Name x imported from module signature Γ has entry θ

```
f()
```

$\vdash \vec{\imath} : \Gamma$

imports-nil

$$\frac{}{\vdash \varepsilon : \varnothing}$$

import

$$\frac{q' = x \cdot \vec{x} \qquad q \not\prec q' \qquad \vdash_{\text{load}} q' : \Delta \qquad q', \text{Mod}(q', \Delta) \text{ loads-as } \theta \qquad \vdash \vec{\imath} : \Gamma}{\vdash \texttt{import}\ q' \cdot \vec{\imath} : \{x{:}\,\theta\} \lhd \Gamma}$$

from-import

$$\frac{\vdash_{\text{load}} q' : \Delta \qquad \vdash_{\text{load}} q'' : \Delta_{q''} \quad (\forall q'' \prec q',\ q'' \not\preceq q) \qquad \Delta, x_i \text{ imports } \theta_i \quad (\forall i) \qquad \vdash \vec{\imath} : \Gamma}{\vdash \texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} : \{x_1{:}\,\theta_1, \ldots, x_{n+1}{:}\,\theta_{n+1}\} \lhd \Gamma}$$

(a) Import prefix $\vec{\imath}$ of module q produces bindings Γ

q, θ loads-as θ'

loads-as-simple

$$\frac{}{x, \theta \text{ loads-as } \theta}$$

loads-as-qualified

$$\frac{\vdash_{\text{load}} q : \Gamma' \qquad q, \text{Mod}(q, \Gamma' \lhd \{x{:}\,\theta\}) \text{ loads-as } \theta'}{q.x, \theta \text{ loads-as } \theta'}$$

(b) Reference θ for module q loads as the reference θ' for the root of q, loading the ancestors of q

Fig. 11. Import well-formedness

Because `y` appears on the left-hand side of an assignment in `f`, it is in assigns(b) for the body of `f`, and is therefore introduced at ff at the top of the function. The reference to `y` in `x = y` then fails the well-formedness check, since $\Gamma(\texttt{y}) = \text{ff}$ rather than $\Gamma(\texttt{y}) = \text{tt}$.

3.4.4 Reassignment of captured variables. The exclusion of captured-variable reassignment (§2.1.1) is enforced by a side condition on the seq rule: no variable captured by a statement s may be reassigned in the rest of the block b. In the example of §2.1.1, `x` is captured by the definition of `g` and reassigned by `x = 6`, so the program is rejected. A dataclass declaration counts as an assignment of its class name for these definitions, though re-declaring a class name is prohibited outright (§2.1.5).

The same issue arises when a variable is captured and reassigned in the *same* statement:

```
x = 5
def x():
    return x
x()  # Python: returns the function (late binding); pure: returns 5
```

The side condition on the assign rule rejects programs like this, requiring that the variable being bound is not captured by the expression. The mutual rule, by contrast, binds each f_i in the typing context for every body b_i, so the function names may legitimately appear in captures of those bodies. This is what permits self-recursion (the singleton-region case) and mutual recursion within a region.

3.4.5 Mutual recursion. The mutual rule binds the names $f_1, \ldots, f_{n+1}$ of a mutual region $d_1 \ldots d_{n+1}$ simultaneously. Each body b_i is checked in a context that includes all the f_j as definitely assigned, so any f_j may reference any other f_k within the region (the singleton case $n = 0$ permits self-recursion). The names in a region must be distinct.

Inserting any non-`def` statement between two `def`s splits them into separate regions, breaking the mutual reference:

```
def even(n):
```

$\boxed{\Gamma \vdash \vec{s} : \Delta}$

nil

$$\frac{}{\Gamma \vdash \varepsilon : \varnothing}$$

statement

$$\frac{\Gamma \vdash s : \text{Assigns } \Delta \qquad \Gamma \cdot \Delta \vdash \vec{s} : \Delta' \qquad \text{captures}(s) \cap \text{assigns}(\vec{s}) = \varnothing}{\Gamma \vdash s \cdot \vec{s} : \Delta \cdot \Delta'}$$

class

$$\frac{\nexists C'.\ c{:}C' \in \Gamma \qquad \text{own-fields}(\phi) = \vec{x}\ \text{distinct} \qquad C = \text{Cls}_\Gamma(q.c, \vec{x}, \bot) \qquad \Gamma, c{:}C \vdash \vec{s} : \Delta}{\Gamma \vdash \texttt{@dataclass class } c : \ \phi \cdot \vec{s} : \{c{:}C\} \cdot \Delta}$$

class-extend

$$\frac{\nexists C'.\ c{:}C' \in \Gamma \qquad c'{:}\text{Cls}_\Delta(q.c', \vec{y}, c'') \in \Gamma \qquad \text{own-fields}(\phi) = \vec{x}\ \text{distinct} \qquad \vec{x} \cap \text{fields}(\Gamma(c')) = \varnothing \qquad C = \text{Cls}_\Gamma(q.c, \vec{x}, c') \qquad \Gamma, c{:}C \vdash \vec{s} : \Delta}{\Gamma \vdash \texttt{@dataclass class } c(c') : \ \phi \cdot \vec{s} : \{c{:}C\} \cdot \Delta}$$

(a) Statements $\vec{s}$ of module q are well-formed in Γ

$\boxed{\vdash_{\text{load}} q : \Gamma}$

predefined

$$\frac{q \text{ predefined}}{\vdash_{\text{load}} q : \Gamma_q}$$

module

$$\frac{\mathcal{M}(q) = \vec{\imath}\,\vec{s} \qquad \vdash \vec{\imath} : \Gamma \qquad \Gamma \vdash (\texttt{__name__} = \text{name}(q)) \cdot \vec{s} : \Delta \qquad (\text{dom}(\Gamma) \cup \text{dom}(\Delta)) \cap \text{dom}(\text{submods}(q)) = \varnothing}{\vdash_{\text{load}} q : \text{submods}(q) \cdot \Delta}$$

$\boxed{\vdash \mathcal{M}}$

program

$$\frac{\vdash_{\text{load}} \texttt{__main__} : \Gamma}{\vdash \mathcal{M}}$$

(b) Module and program well-formedness

Fig. 12. Module well-formedness

```
if a < b:
   comp = "smaller"
else:
   comp = "bigger"
return comp
```

```
if a < b:
   return "smaller"
else:
   comp = "bigger"
return comp
```

```
    if n == 0:
        return True
    return odd(n - 1)

x = even(0)  # intervening statement separates the regions
```

```
def foo():
  z = 6
  if b:
    x = 3
    y = x + 1
  else:
    y = 0
  return z
```

(a) Branch-local definition of `x`

```
def foo():
  x = 6
  if b:
    x = 3
    y = x + 1
  else:
    y = 0
  return x # error: partially defined
```

(b) Now with use of `x` after the conditional

```
def odd(n):
    if n == 0:
        return False
    return even(n - 1)
```

This program is rejected statically: `even` forms a singleton region in which `odd` is not yet bound, so the reference to `odd` in its body fails the `var` rule. Python accepts it: the call `even(0)` does not actually reach `odd`, and Python's late binding means `odd` is only looked up when needed.

Each branch of a conditional may contain its own mutual regions, subject to the usual definite assignment rules. The following is well-formed because both branches define both names:

```
b = True

if b:
    def f():
        return g()
    def g():
        return "via mutual region"
else:
    def f():
        return "f independent"
    def g():
        return "g independent"

print(f())
print(g())
```

A region containing two definitions of the same name is rejected directly by the `mutual` rule's distinct-names side condition:

```
def g():
    return 0

def f():
    return g()
def g():     # error: g defined twice in the same region
    return 1
```

(Python accepts this; late binding makes `f`'s call to `g` resolve to the second `def g`.)

Wrapping the second `def f; def g` pair in a conditional sidesteps the duplicate-name issue: the outer `def g` and the inner `def g` are now in different blocks, so they no longer fall in the same region. One might still expect a captured-variable problem, since the inner `def g` could be seen as rebinding the outer `g` that `f` captures. No such problem arises:

```
def g():
    return 0

if b:
    def f():
        return g()
    def g():
        return 1
```

The inner `def f; def g` form a branch-local mutual region, so `f`'s body resolves `g` to its sibling rather than to the outer `g`. The two are bound simultaneously, so `f` cannot be called before the inner `g` exists, and the value `f` sees is the same as the value Python's late binding finds at call time.

3.4.6 Pattern variables escape the match. A variable bound by a `case` pattern is in scope after the `match` statement, just as a variable assigned inside a conditional branch is in scope after the conditional. Python treats matching this way; PurePy follows.

```
v = 5
match v:
    case x:
        pass
print(x)    # prints 5
```

Here the catch-all `case x:` binds `x` in the body's scope, and the binding survives into the enclosing scope. Definite assignment after the match follows the same merge logic as a conditional: if every case binds the variable (or returns), it is definitely assigned afterwards; otherwise it is not (see also §3.4.2).

3.4.7 Unreachable code. A non-empty sequence of statements returns only if the last statement returns, and all preceding statements assign. Thus the following is illegal:

```
def foo(x):
  return x + 1
  return x    # error: unreachable
```

3.4.8 Conditionals without `else` clauses. An `if` statement without an `else` clause is equivalent to an `if` statement with an `else` clause whose body is a `pass` statement; such a statement never returns.

3.4.9 Implicit `None`. A `return` statement without an expression is equivalent to `return None`.

3.4.10 Implicit `return`. A function whose body s assigns rather than returns is equivalent to a function whose body is s followed by `return None`. This ensures every function returns, even if it is not explicitly specified.

3.4.11 `assert` without a message. An `assert e` statement without a message is equivalent to `assert e, ""`.

$$
\begin{array}{rcll}
v & ::= & \ell & \text{(Literal)} \\
 & | & [v_1, \ldots, v_n] & \text{(List)} \\
 & | & (v_1, \ldots, v_n) & \text{(Tuple)} \\
 & | & \mathsf{Lam}_\Gamma(\rho, \vec{x}, e) & \text{(LambdaClosure)} \\
 & | & \mathsf{Def}_\Gamma(\rho, \vec{d}, i) & \text{(DefClosure)} \\
 & | & \mathsf{Obj}_\Gamma(C, \rho) & \text{(Object)} \\
 & | & \mathsf{Mod}(q) & \text{(Module stub)} \\
 & | & \mathsf{Mod}(q, \rho) & \text{(Loaded module)} \\
 & | & \mathsf{Cls}_\Gamma(q.c, \vec{x}, c') & \text{(Class)} \\
\\
\rho & ::= & \{x_1 : v_1, \ldots, x_n : v_n\} & \text{(Environment)} \\
\\
r & ::= & \mathsf{assigns}\ \rho & \\
 & | & \mathsf{returns}\ v &
\end{array}
$$

Fig. 15. Values, environments and evaluation results

4 Operational semantics

Values, environments, and evaluation results are given in Figure 15. For a statement s, the result assigns ρ represents *normal completion* of s, with accumulated environment ρ; the result returns v represents *abrupt completion* of s with return value v. Composing statements sequentially composes their results according to the following partial monoid, analogous to sequential composition of result types:

$$\mathsf{assigns}\ \rho \cdot \mathsf{assigns}\ \rho' = \mathsf{assigns}\ (\rho \cdot \rho')$$
$$\mathsf{assigns}\ \rho \cdot \mathsf{returns}\ v = \mathsf{returns}\ v$$

with assigns $\varnothing$ as left and right unit.

A value is well-formed at a context entry (Figure 16): ordinary values at $\mathtt{tt}$, module references and class entries at the corresponding entry. An environment is well-formed at a context that binds its variables at corresponding entries.

Module and program evaluation are given in Figure 19; import evaluation in Figure 18; importing names from a loaded module in Figure 17. Extend (Definition 4) extends to module values by reading its equations with a loaded reference $\mathsf{Mod}(q, \rho)$ carrying an environment in place of a context. The loading judgement $q \Rightarrow_{\mathsf{load}} \rho$ evaluates the import prefix of q and then its statements, mirroring the well-formedness judgement $\vdash_{\mathsf{load}} q : \Gamma$ (Figure 12); as with the static rules, the mutual recursion between loading and evaluation is grounded by program well-formedness, which rules out import cycles (§3.2). Loading takes a single module; the ancestors of an imported module are loaded by the loads-as judgement for an import, and directly as premises of eval-from-import for a from-import.

The correspondence is exact: if $\vdash_{\mathsf{load}} q : \Gamma$ and $q \Rightarrow_{\mathsf{load}} \rho$ then $\vdash \rho : \Gamma$ (Figure 16). Since imports precede statements, every module a statement can reference is loaded before the statement runs, and since loading has no observable effects, reloading a module is unobservable; the semantics therefore needs no cache of loaded modules. These rules use the environment $\rho_{\texttt{builtins}}$ binding the names of Figure 6. A program evaluates to its exit status: a run whose main module loads successfully yields 0 (rule eval-program).

Block evaluation $\rho, b \Rightarrow r$ is given in Figure 20. Function definitions delimit the scope of sequential-composition effects, so as black boxes functions are always pure.

Expression evaluation $\rho, e \Rightarrow v$ is given in Figure 22. The strict operators are given by partial metafunctions $\hat{\odot}$ and $\hat{\ominus}$, fixed by the implementation over the primitive values. Where one of these is undefined, as for division by zero or a subscript out of range, the expression has no derivation.

$\vdash v : \theta$

$$\textsc{val-lit}\ \frac{}{\vdash \ell : \mathrm{tt}} \qquad \textsc{val-list}\ \frac{\vdash v_i : \mathrm{tt} \quad (\forall i)}{\vdash [v_1, \ldots, v_n] : \mathrm{tt}} \qquad \textsc{val-tuple}\ \frac{\vdash v_i : \mathrm{tt} \quad (\forall i)}{\vdash (v_1, \ldots, v_n) : \mathrm{tt}} \qquad \textsc{val-lambda}\ \frac{\vdash \rho : \Gamma \qquad \Gamma \cdot \mathrm{tt}(\vec{x}) \vdash e}{\vdash \mathrm{Lam}_\Gamma(\rho, \vec{x}, e) : \mathrm{tt}}$$

$$\textsc{val-def}\ \frac{\vdash \rho : \Gamma \qquad \Gamma \cdot \mathrm{tt}(\vec{f} \cdot \vec{x}_j) \cdot \mathrm{ff}(\mathrm{assigns}(b_j) \setminus \vec{x}_j) \vdash b_j : R_j \quad (\forall j) \qquad \vec{f}\ \text{distinct}}{\vdash \mathrm{Def}_\Gamma(\rho, \texttt{def}\ f_1(\vec{x}_1) :\ b_1 \cdot \ldots \cdot \texttt{def}\ f_{n+1}(\vec{x}_{n+1}) :\ b_{n+1}, i) : \mathrm{tt}} \qquad \textsc{val-obj}\ \frac{\vdash C : C \qquad \vdash \rho : \Gamma}{\vdash \mathrm{Obj}_\Gamma(C, \rho) : \mathrm{tt}}$$

$$\textsc{val-mod}\ \frac{q \in \mathrm{dom}(\mathcal{M})}{\vdash \mathrm{Mod}(q) : \mathrm{Mod}(q)} \qquad \textsc{val-mod-loaded}\ \frac{\vdash \rho : \Gamma}{\vdash \mathrm{Mod}(q, \rho) : \mathrm{Mod}(q, \Gamma)} \qquad \textsc{val-class}\ \frac{}{\vdash \mathrm{Cls}_\Gamma(q.c, \vec{x}, \bot) : \mathrm{Cls}_\Gamma(q.c, \vec{x}, \bot)}$$

$$\textsc{val-class-extend}\ \frac{c' : C' \in \Gamma}{\vdash \mathrm{Cls}_\Gamma(q.c, \vec{x}, c') : \mathrm{Cls}_\Gamma(q.c, \vec{x}, c')}$$

$\vdash \rho : \Gamma$

$$\textsc{env-nil}\ \frac{}{\vdash \varnothing : \varnothing} \qquad \textsc{env-cons}\ \frac{\vdash v : \theta \qquad \vdash \rho : \Gamma}{\vdash \{x : v\} \cdot \rho : \{x : \theta\} \cdot \Gamma}$$

Fig. 16. Well-formed values and environments

ρ, x imports v

$$\textsc{eval-imp-member}\ \frac{x : v \in \rho \qquad v \neq \mathrm{Mod}(q') \quad (\forall q')}{\rho, x \text{ imports } v} \qquad \textsc{eval-imp-submodule}\ \frac{x : \mathrm{Mod}(q') \in \rho \qquad q' \Rightarrow_{\mathrm{load}} \rho'}{\rho, x \text{ imports } \mathrm{Mod}(q', \rho')}$$

Fig. 17. Name x imported from module environment ρ has value v

The operators `and` and `or` evaluate their second operand only when the first leaves the result open, and a conditional expression requires its condition to be `True` or `False`, as eval-if does. A comprehension evaluates its body once for each binding its qualifiers produce, and concatenates the results; a generator draws from a list, because PurePy has no general iteration protocol. The pattern-matching judgement $\rho, p, v \rightsquigarrow \rho'$ (Figure 24) is a deterministic partial function; we write $\rho, p, v \not\rightsquigarrow$ (with no right-hand side) for $\nexists \rho'.\ \rho, p, v \rightsquigarrow \rho'$, i.e. matching is undefined. A constructor pattern matches any object whose class has the pattern's class among its ancestors; sub-patterns bind the fields of the pattern's class. An object records its class entry, whose qualified name is the identity of the class: since a module declares each class name at most once, the entry comparison in pat-constr amounts to a comparison of qualified names.

$\vec{\imath} \Rightarrow \rho$

eval-imports-nil

$$\frac{}{\varepsilon \Rightarrow \varnothing}$$

eval-import

$$\frac{q' = x \cdot \vec{x} \qquad q' \Rightarrow_{\text{load}} \rho \qquad q', \text{Mod}(q', \rho) \text{ loads-as } v \qquad \vec{\imath} \Rightarrow \rho'}{\texttt{import}\ q' \cdot \vec{\imath} \Rightarrow \{x{:}\, v\} \lhd \rho'}$$

eval-from-import

$$\frac{q' \Rightarrow_{\text{load}} \rho \qquad q'' \Rightarrow_{\text{load}} \rho_{q''} \qquad (\forall q'' \prec q',\ q'' \not\preceq q) \qquad \rho, x_i \text{ imports } v_i \quad (\forall i) \qquad \vec{\imath} \Rightarrow \rho''}{\texttt{from}\ q'\ \texttt{import}\ x_1, \ldots, x_{n+1} \cdot \vec{\imath} \Rightarrow \{x_1{:}\, v_1, \ldots, x_{n+1}{:}\, v_{n+1}\} \lhd \rho''}$$

(a) Import prefix $\vec{\imath}$ of module q produces bindings ρ

q, v loads-as v'

eval-loads-as-simple

$$\frac{}{x, v \text{ loads-as } v}$$

eval-loads-as-qualified

$$\frac{q \Rightarrow_{\text{load}} \rho \qquad q, \text{Mod}(q, \rho \lhd \{x{:}\, v\}) \text{ loads-as } v'}{q.x, v \text{ loads-as } v'}$$

(b) Reference v for module q loads as the reference v' for the root of q, loading the ancestors of q

Fig. 18. Import evaluation

$\rho, \vec{s} \Rightarrow \rho'$

eval-nil

$$\frac{}{\rho, \varepsilon \Rightarrow \varnothing}$$

eval-statement

$$\frac{\rho, s \Rightarrow \text{assigns } \rho' \qquad \rho \cdot \rho', \vec{s} \Rightarrow \rho''}{\rho, s \cdot \vec{s} \Rightarrow \rho' \cdot \rho''}$$

eval-class

$$\frac{C = \text{Cls}_\Gamma(q.c, \text{own-fields}(\phi), \bot) \qquad \rho \cdot \{c{:}\, C\}, \vec{s} \Rightarrow \rho'}{\rho, \texttt{@dataclass class}\ c:\ \phi \cdot \vec{s} \Rightarrow \{c{:}\, C\} \cdot \rho'}$$

eval-class-extend

$$\frac{C = \text{Cls}_\Gamma(q.c, \text{own-fields}(\phi), c') \qquad \rho \cdot \{c{:}\, C\}, \vec{s} \Rightarrow \rho'}{\rho, \texttt{@dataclass class}\ c(c'):\ \phi \cdot \vec{s} \Rightarrow \{c{:}\, C\} \cdot \rho'}$$

(a) Statements $\vec{s}$ of module q evaluate under environment $\rho : \Gamma$, producing bindings ρ'

$q \Rightarrow_{\text{load}} \rho$

eval-predefined

$$\frac{q \text{ predefined}}{q \Rightarrow_{\text{load}} \rho_q}$$

load

$$\frac{\mathcal{M}(q) = \vec{\imath}\, \vec{s} \qquad \vec{\imath} \Rightarrow \rho \qquad \rho, (\texttt{__name__} = \text{name}(q)) \cdot \vec{s} \Rightarrow \rho'}{q \Rightarrow_{\text{load}} \text{submods}(q) \cdot \rho'}$$

$\mathcal{M} \Rightarrow n$

eval-program

$$\frac{\texttt{__main__} \Rightarrow_{\text{load}} \rho}{\mathcal{M} \Rightarrow 0}$$

(b) Module loading and program evaluation

Fig. 19. Module and program evaluation

$\rho, b \Rightarrow r$

eval-pass

$$\frac{}{\rho, \texttt{pass} \Rightarrow \text{assigns } \varnothing}$$

eval-assign

$$\frac{\rho, e \Rightarrow v}{\rho, x = e \Rightarrow \text{assigns } \{x : v\}}$$

eval-expr-stmt

$$\frac{\rho, e \Rightarrow v}{\rho, e \Rightarrow \text{assigns } \varnothing}$$

eval-assert

$$\frac{\rho, e \Rightarrow v \qquad v \neq \texttt{False}}{\rho, \texttt{assert } e \Rightarrow \text{assigns } \varnothing}$$

eval-assert-msg

$$\frac{\rho, e \Rightarrow v \qquad v \neq \texttt{False}}{\rho, \texttt{assert } e, e' \Rightarrow \text{assigns } \varnothing}$$

eval-return

$$\frac{\rho, e \Rightarrow v}{\rho, \texttt{return } e \Rightarrow \text{returns } v}$$

eval-def

$$\frac{d_i = \texttt{def } f_i(\vec{x}_i) : \ b_i \quad (\forall i) \qquad v_i = \text{Def}_\Gamma(\rho, \vec{d}, i) \quad (\forall i)}{\rho, \vec{d} \Rightarrow \text{assigns } \{\vec{f} : \vec{v}\}}$$

eval-if

$$\frac{\rho, e_j \Rightarrow \texttt{False} \quad (\forall j < i) \qquad \rho, e_i \Rightarrow \texttt{True} \qquad \rho, b_i \Rightarrow r}{\rho, \texttt{if } e_1 : \ b_1 \ \texttt{elif } e_2 : \ b_2 \ \ldots \ \texttt{elif } e_{n+1} : \ b_{n+1} \ \texttt{else} : \ b_{n+2} \Rightarrow r}$$

eval-else

$$\frac{\rho, e_j \Rightarrow \texttt{False} \quad (\forall j \in \{1, \ldots, n+1\}) \qquad \rho, b_{n+2} \Rightarrow r}{\rho, \texttt{if } e_1 : \ b_1 \ \texttt{elif } e_2 : \ b_2 \ \ldots \ \texttt{elif } e_{n+1} : \ b_{n+1} \ \texttt{else} : \ b_{n+2} \Rightarrow r}$$

eval-match

$$\frac{\rho, e \Rightarrow v \qquad \text{dispatch}(\rho, v, \texttt{case } p_1 : b_1 \cdot \ldots \cdot \texttt{case } p_{n+1} : b_{n+1}) = (\rho', b) \qquad \rho \cdot \rho', b \Rightarrow r}{\rho, \texttt{match } e : \texttt{case } p_1 : b_1 \ \ldots \ \texttt{case } p_{n+1} : b_{n+1} \Rightarrow r}$$

eval-cons

$$\frac{\rho, s \Rightarrow \text{assigns } \rho' \qquad \rho \cdot \rho', b \Rightarrow r}{\rho, s \cdot b \Rightarrow \text{assigns } \rho' \cdot r}$$

Fig. 20. Block b evaluates to result r under environment $\rho : \Gamma$

$$\begin{aligned} \text{unwrap}(\text{returns } v) &= v \\ \text{unwrap}(\text{assigns } \rho) &= \texttt{None} \end{aligned}$$

$$\begin{aligned} \text{env}(\rho, x) &= \rho' && \text{where } \rho(x) = \text{Mod}(q', \rho') \\ \text{env}(\rho, q.x) &= \rho' && \text{where } \text{env}(\rho, q)(x) = \text{Mod}(q', \rho') \end{aligned}$$

$$\begin{aligned} \text{resolve-class}(\rho, c) &= \rho(c) \\ \text{resolve-class}(\rho, q.c) &= \text{env}(\rho, q)(c) \end{aligned}$$

$$\begin{aligned} \text{dispatch}(\rho, v, \varepsilon) &= (\varnothing, \texttt{pass}) \\ \text{dispatch}(\rho, v, (\texttt{case } p : b) \cdot \vec{c}) &= (\rho', b) && \text{if } \rho, p, v \rightsquigarrow \rho' \\ \text{dispatch}(\rho, v, (\texttt{case } p : b) \cdot \vec{c}) &= \text{dispatch}(\rho, v, \vec{c}) && \text{if } \rho, p, v \not\rightsquigarrow \end{aligned}$$

Fig. 21. Metafunctions used by block and expression evaluation

$\rho, e \Rightarrow v$

eval-var
$$\frac{x{:}v \in \rho}{\rho, x \Rightarrow v}$$

eval-var-builtin
$$\frac{x \notin \mathrm{dom}(\rho) \qquad x{:}v \in \rho_{\texttt{builtins}}}{\rho, x \Rightarrow v}$$

eval-literal
$$\frac{}{\rho, \ell \Rightarrow \ell}$$

eval-lambda
$$\frac{}{\rho, \texttt{lambda}\ x_1, \ldots, x_n{:}\ e \Rightarrow \mathrm{Lam}_\Gamma(\rho, \vec{x}, e)}$$

eval-call-lambda
$$\frac{\rho, e \Rightarrow \mathrm{Lam}_\Delta(\rho', \vec{x}, e') \qquad \rho, \vec{e} \Rightarrow \vec{u} \qquad \rho' \cdot \{\vec{x}{:}\vec{u}\}, e' \Rightarrow v}{\rho, e(e_1, \ldots, e_n) \Rightarrow v}$$

eval-call-def
$$\frac{\begin{array}{c}\rho, e \Rightarrow \mathrm{Def}_\Delta(\rho', \vec{d}, j) \\ d_i = \texttt{def}\ f_i(\vec{x}_i) :\ b_i \quad (\forall i) \qquad v_i = \mathrm{Def}_\Delta(\rho', \vec{d}, i) \quad (\forall i) \qquad \rho, \vec{e} \Rightarrow \vec{u} \qquad \rho' \cdot \{\vec{f}{:}\vec{v}\} \cdot \{\vec{x}_j{:}\vec{u}\}, b_j \Rightarrow r\end{array}}{\rho, e(e_1, \ldots, e_n) \Rightarrow \mathrm{unwrap}(r)}$$

eval-attr-module
$$\frac{\rho, e \Rightarrow \mathrm{Mod}(q', \rho') \qquad x{:}v \in \rho'}{\rho, e.x \Rightarrow v}$$

eval-attr-object
$$\frac{\rho, e \Rightarrow \mathrm{Obj}_\Delta(C, \rho')}{\rho, e.x \Rightarrow \rho'(x)}$$

eval-constr
$$\frac{\rho, e_i \Rightarrow v_i \quad (\forall i) \qquad \text{resolve-class}(\rho, q) = C \qquad \text{fields}(C) = x_1, \ldots, x_n, x_{n+1}, \ldots, x_{n+m}}{\rho, q(e_1, \ldots, e_n, y_1{=}e_{n+1}, \ldots, y_m{=}e_{n+m}) \Rightarrow \mathrm{Obj}_\Gamma(C, \{x_1{:}v_1, \ldots, x_n{:}v_n, y_1{:}v_{n+1}, \ldots, y_m{:}v_{n+m}\})}$$

eval-list
$$\frac{\rho, e_i \Rightarrow v_i \quad (\forall i)}{\rho, [e_1, \ldots, e_n] \Rightarrow [v_1, \ldots, v_n]}$$

eval-tuple
$$\frac{\rho, e_i \Rightarrow v_i \quad (\forall i)}{\rho, (e_1, \ldots, e_n) \Rightarrow (v_1, \ldots, v_n)}$$

eval-subscript
$$\frac{\rho, e \Rightarrow [v_1, \ldots, v_m] \qquad \rho, e' \Rightarrow n \qquad 0 \le n < m}{\rho, e[e'] \Rightarrow v_{n+1}}$$

eval-subscript-neg
$$\frac{\rho, e \Rightarrow [v_1, \ldots, v_m] \qquad \rho, e' \Rightarrow n \qquad -m \le n < 0}{\rho, e[e'] \Rightarrow v_{m+n+1}}$$

eval-binop
$$\frac{\rho, e \Rightarrow v \qquad \rho, e' \Rightarrow v' \qquad \hat{\odot}(v, v') = u}{\rho, e \odot e' \Rightarrow u}$$

eval-unary-op
$$\frac{\rho, e \Rightarrow v \qquad \hat{\ominus}(v) = u}{\rho, \ominus e \Rightarrow u}$$

eval-and-false
$$\frac{\rho, e \Rightarrow \texttt{False}}{\rho, e\ \texttt{and}\ e' \Rightarrow \texttt{False}}$$

eval-and-true
$$\frac{\rho, e \Rightarrow \texttt{True} \qquad \rho, e' \Rightarrow v}{\rho, e\ \texttt{and}\ e' \Rightarrow v}$$

eval-or-true
$$\frac{\rho, e \Rightarrow \texttt{True}}{\rho, e\ \texttt{or}\ e' \Rightarrow \texttt{True}}$$

eval-or-false
$$\frac{\rho, e \Rightarrow \texttt{False} \qquad \rho, e' \Rightarrow v}{\rho, e\ \texttt{or}\ e' \Rightarrow v}$$

eval-cond-true
$$\frac{\rho, e \Rightarrow \texttt{True} \qquad \rho, e_1 \Rightarrow v}{\rho, e_1\ \texttt{if}\ e\ \texttt{else}\ e_2 \Rightarrow v}$$

eval-cond-false
$$\frac{\rho, e \Rightarrow \texttt{False} \qquad \rho, e_2 \Rightarrow v}{\rho, e_1\ \texttt{if}\ e\ \texttt{else}\ e_2 \Rightarrow v}$$

eval-list-comp
$$\frac{\rho, \vec{g}, e \Rightarrow v_1 \ldots v_m}{\rho, [\ e\ g_1\ \ldots\ g_n] \Rightarrow [v_1, \ldots, v_m]}$$

Fig. 22. Expression e evaluates to value v under environment $\rho : \Gamma$

5 Reference implementation

A reference well-formedness checker, written in Python over the `ast` module, decides membership of PurePy and reports why a program is rejected. This section describes the checker and its correspondence with the well-formedness rules of §3.

$\rho, \vec{g}, e \Rightarrow \vec{v}$

eval-quals-nil

$$\frac{\rho, e \Rightarrow v}{\rho, \varepsilon, e \Rightarrow v}$$

eval-qual-guard-true

$$\frac{\rho, e' \Rightarrow \texttt{True} \qquad \rho, \vec{g}, e \Rightarrow \vec{v}}{\rho, \texttt{if } e' \cdot \vec{g}, e \Rightarrow \vec{v}}$$

eval-qual-guard-false

$$\frac{\rho, e' \Rightarrow \texttt{False}}{\rho, \texttt{if } e' \cdot \vec{g}, e \Rightarrow \varepsilon}$$

eval-qual-generator

$$\frac{\rho, e' \Rightarrow [u_1, \ldots, u_n] \qquad \rho \cdot \{x{:}\, u_i\}, \vec{g}, e \Rightarrow \vec{v}_i \quad (\forall i)}{\rho, \texttt{for } x \texttt{ in } e' \cdot \vec{g}, e \Rightarrow \vec{v}_1 \cdot \ldots \cdot \vec{v}_n}$$

Fig. 23. Comprehension qualifiers $\vec{g}$ with body e produce values $\vec{v}$

$\rho, p, v \rightsquigarrow \rho'$

pat-lit

$$\frac{}{\rho, \ell, \ell \rightsquigarrow \varnothing}$$

pat-neg-lit

$$\frac{}{\rho, -n, -n \rightsquigarrow \varnothing}$$

pat-var

$$\frac{}{\rho, x, v \rightsquigarrow \{x{:}\, v\}}$$

pat-wild

$$\frac{}{\rho, _, v \rightsquigarrow \varnothing}$$

pat-list

$$\frac{\rho, p_i, v_i \rightsquigarrow \rho_i \quad (\forall i)}{\rho, [p_1, \ldots, p_n], [v_1, \ldots, v_n] \rightsquigarrow \rho_1 \uplus \ldots \uplus \rho_n}$$

pat-tuple

$$\frac{\rho, p_i, v_i \rightsquigarrow \rho_i \quad (\forall i)}{\rho, (p_1, \ldots, p_n), (v_1, \ldots, v_n) \rightsquigarrow \rho_1 \uplus \ldots \uplus \rho_n}$$

pat-as

$$\frac{\rho, p, v \rightsquigarrow \rho'}{\rho, p \texttt{ as } x, v \rightsquigarrow \rho' \uplus \{x{:}\, v\}}$$

pat-constr

$$\frac{\begin{array}{c}\text{resolve-class}(\rho, q) = C \qquad C \in \text{ancestors}(C') \qquad \text{fields}(C) = x_1, \ldots, x_n, x_{n+1}, \ldots, x_{n+m} \\ \rho, p_i, \rho''(x_i) \rightsquigarrow \rho_i \quad (\forall i.\, 1 \le i \le n) \qquad \rho, p_{n+j}, \rho''(y_j) \rightsquigarrow \rho_{n+j} \quad (\forall j.\, 1 \le j \le m)\end{array}}{\rho, q(p_1, \ldots, p_n, y_1{=}p_{n+1}, \ldots, y_m{=}p_{n+m}), \text{Obj}_{\Gamma'}(C', \rho'') \rightsquigarrow \rho_1 \uplus \ldots \uplus \rho_{n+m}}$$

Fig. 24. Pattern p matches value v under environment ρ, producing bindings ρ'

5.1 Conformance suite

A conformance test suite pairs each test with the verdict Python itself gives, and runs on CPython, PyPy, GraalPy, and Pyodide. [**TODO:** Diagram showing the various kinds of test.]

5.2 Mechanisation in Isabelle

[**TODO:** Mechanisation of the well-formedness rules and operational semantics in Isabelle (#64).]

6 Extensions

Beyond the core, PurePy is designed to grow through extensions that add expressiveness while preserving purity. This section sketches two, each drawn from Fluid: partial application and piecewise function definitions. We give an example of each. [**TODO:** Formally define or defer to future work]

6.1 Partial application

A function applied to fewer arguments than it declares returns a function of the remaining arguments:

```
def f(x, y):
    return x + y

f(5)(6)  # 11
```

The syntax is valid Python, but Python raises a `TypeError` at run time, because `f(5)` is missing an argument. For partial application to add meaning rather than change it, core PurePy must reject such unsaturated calls, requiring every call to supply the declared arity. On that base, `f(5)` denotes the function that adds `5` to its argument, so `f(5)(6)` evaluates to `11`. This is a semantic extension: existing syntax, given a meaning the core withholds.

6.2 Piecewise function definitions

A function may be defined by several clauses that pattern-match on its arguments, in place of a single definition with an explicit `match`:

```
def length(Nil):
    return 0
def length(Cons(x, xs)):
    return 1 + length(xs)
```

Python has no way to define a function by pattern-matching on its arguments: `def length(Cons(x, xs))` is a syntax error, since a parameter cannot be a pattern. Core PurePy, a subset of Python, has none either. The extension adds this: the clauses together define `length`, dispatching on the constructor of the argument. Because Python gives the pattern-clause form no meaning of its own, it is a syntactic extension that needs no special notation to distinguish it from an ordinary `def`.

6.3 Explicit variable definitions

In the core a variable is introduced implicitly by its first assignment, and a definite-assignment analysis (§3) tracks whether it is assigned before it is used. This extension makes the introduction explicit, with a `def` declaration for variables as well as functions:

```
def total = 0   # define and assign

def n           # define, assignment deferred
if b:
    n = 1
else:
    n = 2
return total + n
```

`def total = 0` declares `total` and assigns it at once, whereas `def n` declares `n` with its assignment deferred, to be supplied later under the existing definite-assignment rules, so `n` is usable only once every path has assigned it. Python has no such form: `def` introduces a function, so a variable `def` is a syntax error, and core PurePy, a subset of Python, has none either. The implicit definitions of the core are the case in which every variable is introduced by `def x = e`, so the core desugars into the explicit form.

7 Case studies

This section reports two research languages for scientific computing that share PurePy's Python-like surface: Fortl and Fluid. [**TODO:** Is there a way to do this without deanonymising the paper?]

7.1 Fortl

Fortl is a statically-typed research language for scientific and numerical computing, with a Python-like syntax. Its distinctive feature is a system of graded numerical types: types are indexed and carry structure, so domain concepts such as units of measure and physical quantities are expressed in the types and enforced by the type checker. A length divided by a time is typed as a speed, and a program that mixes incompatible units is rejected. Fortl aims to let code state the meaning, intent, and limitations of a scientific model, and to make its predictions reproducible and connected to their abstract description. [**TODO:** Confirm and describe the relationship to PurePy]

7.2 Fluid

Fluid is a pure, Pythonic, provenance-aware functional language for transparent research outputs. It is built on PurePy, adding a runtime that records fine-grained dependencies between a computation's inputs and its outputs, so that a figure or value can be linked back to the data it was computed from. Fluid supports pattern matching, destructuring, and partial application, and is aimed at researchers who need both computation and verifiable transparency. As a case study it exercises PurePy as the functional core beneath a substantial additional runtime, and its surface motivates the partial-application and piecewise extensions of §6.

8 Evaluation

We evaluate PurePy empirically along two axes, both concerning how well AI agents work with the subset.

8.1 Generating PurePy

How effectively can AI agents generate PurePy solutions to standard functional-programming problems? This measures how readily the subset can be targeted from a problem statement alone.

8.2 Translating Python to PurePy

How effectively can AI agents translate existing Python programs into the PurePy subset? [**TODO:** Not sure about this one without libraries being part of the spec too.]

9 Related work

9.1 Formal semantics and verified subsets

Although the syntax of mainstream languages is usually described precisely, few define their behaviour formally, so well-defined subsets have been constructed to equip languages with formal, often mechanised, semantics. Examples include Java [Igarashi et al. 2001], OCaml [Seassau et al. 2025], JavaScript [Bodin et al. 2014; Guha et al. 2010; Park et al. 2015; Politz et al. 2012], PHP [Filaretti and Maffeis 2014], Rust [Jung et al. 2020, 2018], and lower-level languages such as WebAssembly [Watt 2021; Watt et al. 2021, 2023] and LLVM-IR [Zakowski et al. 2021; Zhao et al. 2012]. A notable success is Clight [Blazy and Leroy 2009], a substantial subset of C that underpins a verified compiler [Leroy 2009]; likewise the CakeML project [Kumar et al. 2014] produced a verified implementation of a fragment of ML [Milner et al. 1997], building on earlier mechanisations [Lee et al. 2007].

9.2 Formal semantics of Python

Several efforts formalise Python itself: an embedding [Guth 2013] in the $\mathbb{K}$ framework [Roșu and Șerbănuță 2010], a structural operational semantics [Köhl 2021], a reduction of much of the language to a core calculus [Politz et al. 2013], an executable semantics aimed at optimising compilers [Melançon et al. 2023], and a skeletal semantics of a fragment [Andrieux and Schmitt [n. d.]]. PurePy differs in restricting the language to a well-behaved subset rather than modelling Python in full; relating our semantics to these is future work.

9.3 Language subsets for a purpose

A separate line of work delineates subsets for a particular use. The *Source* family [Anderson et al. 2021] comprises progressively larger subsets of JavaScript for teaching first-year students the foundations of computing, following a JavaScript edition of a classic text [Abelson et al. 2022]; the choice of JavaScript was pragmatic, since industrial relevance weighs heavily in language choice for introductory courses [Simon et al. 2018]. For Python, ChocoPy [Padhye et al. 2019] is a statically-typed subset used as the source language in compiler-construction courses, and Restricted Python (RPython) [Ancona et al. 2007] is a statically typed subset that the PyPy project [Rigo and Pedroni 2006] compiles to common runtimes.

9.4 Functional Python

We are aware of one other functional subset of Python [Sunderraman 2025], intended for teaching; PurePy encloses more of the language, since that work omits control structures such as conditionals. Related efforts sit outside the subset discipline: Coconut[4] is a functional language that compiles to Python, and functional pipelines have been proposed for scientific computation in Python [Zhang et al. 2024]. Although PurePy does not aim to make Python more functional, the community has moved in that direction, from lambda expressions in Python 1.0 to a recent proposal for deeply immutable objects [Johnson et al. 2025]. Typed treatments of effects in functional languages, such as Prism,[5] are relevant to the structured effects PurePy may add in a later release.

10 Conclusions and future work

10.1 PurePy-compatible standard libraries

Scientific computing in Python rests on libraries such as NumPy and Pandas, whose interfaces lean on mutation and indexing. PurePy therefore needs compatible counterparts: libraries, or subsets of existing ones, whose operations are pure. Polars,[6] whose operations return new frames rather than updating in place, suggests the shape such interfaces can take.

10.2 Typed effects

A future version of PurePy may add a typed effect discipline. In a mypy-compatible type system the question is notational, because effect information must fit within annotations that Python and mypy already accept. A return type such as `IO[int]` tracks effects as an ordinary generic type, in the Haskell style, though chaining is awkward in a language of statements without dedicated syntax. `Annotated[int, Console]` (PEP 593) attaches effect metadata that mypy ignores and the PurePy checker can read. Lightest is a decorator: with effects the default, `@pure` marks functions in which printing and other effects are rejected, and unannotated code keeps its current meaning. Under any of these, existing support for printing is not an obstacle, since printing can be reinterpreted as

[4] https://coconut-lang.org
[5] https://www.stephendiehl.com/posts/prism/
[6] https://docs.pola.rs/

a typed output effect or kept native with essentially equivalent semantics, printed output never influencing a result (§2.1). Typed disciplines coexisting with native effects have precedent, from Haskell's untracked `trace` in pure code[7] to Scala's monadic effect libraries alongside the language's native effects.

References

H. Abelson, G.J. Sussman, M. Henz, T. Wrigstad, and J. Sussman. 2022. *Structure and Interpretation of Computer Programs: JavaScript Edition.* MIT Press. https://books.google.co.uk/books?id=mt-MEAAAQBAJ

Davide Ancona, Massimo Ancona, Antonio Cuni, and Nicholas D. Matsakis. 2007. RPython: A Step towards Reconciling Dynamically and Statically Typed OO Languages. In *Proceedings of the 2007 Symposium on Dynamic Languages*. ACM, Montreal Quebec Canada, 53–64. doi:10.1145/1297081.1297091

Boyd Anderson, Martin Henz, Kok-Lim Low, and Daryl Tan. 2021. Shrinking JavaScript for CS1. In *Proceedings of the 2021 ACM SIGPLAN International Symposium on SPLASH-E*. ACM, Chicago IL USA, 87–96. doi:10.1145/3484272.3484970

Martin Andrieux and Alan Schmitt. [n. d.]. Skeletal Semantics of a Fragment of Python. ([n. d.]).

Sandrine Blazy and Xavier Leroy. 2009. Mechanized Semantics for the Clight Subset of the C Language. *Journal of Automated Reasoning* 43, 3 (Oct. 2009), 263–288. doi:10.1007/s10817-009-9148-3

Martin Bodin, Arthur Charguéraud, Daniele Filaretti, Philippa Gardner, Sergio Maffeis, Daiva Naudziuniene, Alan Schmitt, and Gareth Smith. 2014. A Trusted Mechanised JavaScript Specification. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, San Diego California USA, 87–100. doi:10.1145/2535838.2535876

Daniele Filaretti and Sergio Maffeis. 2014. An Executable Formal Semantics of PHP. In *ECOOP 2014 – Object-Oriented Programming*, David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Alfred Kobsa, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Demetri Terzopoulos, Doug Tygar, Gerhard Weikum, and Richard Jones (Eds.). Vol. 8586. Springer Berlin Heidelberg, Berlin, Heidelberg, 567–592. doi:10.1007/978-3-662-44202-9_23

James Gosling, Bill Joy, Guy L. Steele Jr., Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman. 2025. *The Java Language Specification, Java SE 24 Edition.* Oracle. https://docs.oracle.com/javase/specs/jls/se24/html/index.html

Arjun Guha, Claudiu Saftoiu, and Shriram Krishnamurthi. 2010. The Essence of Javascript. In *Proceedings of the 24th European Conference on Object-Oriented Programming (ECOOP'10)*. Springer-Verlag, Maribor, Slovenia, 126–150.

Dwight Guth. 2013. *A Formal Semantics of Python 3.3.* Ph. D. Dissertation. University of Illinois at Urbana-Champaign.

Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. 2001. Featherweight Java: A Minimal Core Calculus for Java and GJ. *ACM Transactions on Programming Languages and Systems* 23, 3 (May 2001), 396–450. doi:10.1145/503502.503505

Matthew Johnson, Matthew Parkinson, Sylvan Clebsch, Fridtjof Stoldt, and Tobias Wrigstad. 2025. PEP 795 – Deep Immutability for Efficient Sharing and Concurrency Safety. https://pep-previews--4468.org.readthedocs.build/pep-0795/

Ralf Jung, Hoang-Hai Dang, Jeehoon Kang, and Derek Dreyer. 2020. Stacked Borrows: An Aliasing Model for Rust. *Proceedings of the ACM on Programming Languages* 4, POPL (Jan. 2020), 1–32. doi:10.1145/3371109

Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2018. RustBelt: Securing the Foundations of the Rust Programming Language. *Proceedings of the ACM on Programming Languages* 2, POPL (Jan. 2018), 1–34. doi:10.1145/3158154

Maximilian A. Köhl. 2021. An Executable Structural Operational Formal Semantics for Python. arXiv:2109.03139 [cs] doi:10.48550/arXiv.2109.03139

Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: A Verified Implementation of ML. *ACM SIGPLAN Notices* 49, 1 (Jan. 2014), 179–191. doi:10.1145/2578855.2535841

Daniel K. Lee, Karl Crary, and Robert Harper. 2007. Towards a Mechanized Metatheory of Standard ML. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, Nice France, 173–184. doi:10.1145/1190216.1190245

Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (July 2009), 107–115. doi:10.1145/1538788.1538814

Olivier Melançon, Marc Feeley, and Manuel Serrano. 2023. An Executable Semantics for Faster Development of Optimizing Python Compilers. In *Proceedings of the 16th ACM SIGPLAN International Conference on Software Language Engineering*. ACM, Cascais Portugal, 15–28. doi:10.1145/3623476.3623529

Robin Milner, Robert Harper, David MacQueen, and Mads Tofte. 1997. *The Definition of Standard ML* (2 ed.). The MIT Press. doi:10.7551/mitpress/2319.001.0001

[7] https://hackage.haskell.org/package/base/docs/Debug-Trace.html

Rohan Padhye, Koushik Sen, and Paul N. Hilfinger. 2019. ChocoPy: A Programming Language for Compilers Courses. In *Proceedings of the 2019 ACM SIGPLAN Symposium on SPLASH-E*. ACM, Athens Greece, 41–45. doi:10.1145/3358711.3361627

Daejun Park, Andrei Stefănescu, and Grigore Roşu. 2015. KJS: A Complete Formal Semantics of JavaScript. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Portland OR USA, 346–356. doi:10.1145/2737924.2737991

Joe Gibbs Politz, Matthew J. Carroll, Benjamin S. Lerner, Justin Pombrio, and Shriram Krishnamurthi. 2012. A Tested Semantics for Getters, Setters, and Eval in JavaScript. In *Proceedings of the 8th Symposium on Dynamic Languages*. ACM, Tucson Arizona USA, 1–16. doi:10.1145/2384577.2384579

Joe Gibbs Politz, Alejandro Martinez, Mae Milano, Sumner Warren, Daniel Patterson, Junsong Li, Anand Chitipothu, and Shriram Krishnamurthi. 2013. Python: The Full Monty. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications*. ACM, Indianapolis Indiana USA, 217–232. doi:10.1145/2509136.2509536

Armin Rigo and Samuele Pedroni. 2006. PyPy's Approach to Virtual Machine Construction. In *Companion to the 21st ACM SIGPLAN Symposium on Object-oriented Programming Systems, Languages, and Applications*. ACM, Portland Oregon USA, 944–953. doi:10.1145/1176617.1176753

Grigore Roşu and Traian Florin Şerbănuţă. 2010. An Overview of the K Semantic Framework. *The Journal of Logic and Algebraic Programming* 79, 6 (Aug. 2010), 397–434. doi:10.1016/j.jlap.2010.03.012

Remy Seassau, Irene Yoon, Jean-Marie Madiot, and François Pottier. 2025. Formal Semantics and Program Logics for a Fragment of OCaml. *Proceedings of the ACM on Programming Languages* 9, ICFP (Aug. 2025), 128–159. doi:10.1145/3747509

Simon, Raina Mason, Tom Crick, James H. Davenport, and Ellen Murphy. 2018. Language Choice in Introductory Programming Courses at Australasian and UK Universities. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*. ACM, Baltimore Maryland USA, 852–857. doi:10.1145/3159450.3159547

Rajshekhar Sunderraman. 2025. Functional Python Programming in Introductory Computer Science Courses. arXiv:2512.03492 [cs] doi:10.48550/arXiv.2512.03492

W. G. van Hoorn and B. van Rootselaar. 1967. Fundamental Notions in the Theory of Seminearrings. *Compositio Mathematica* 18, 1-2 (1967), 65–78. https://www.numdam.org/item/CM_1967__18_1-2_65_0/

Conrad Watt. 2021. *Mechanising and Evolving the Formal Semantics of WebAssembly: The Web’s New Low-Level Language*. Ph. D. Dissertation. Apollo - University of Cambridge Repository. doi:10.17863/CAM.76476

Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, and Philippa Gardner. 2021. Two Mechanisations of WebAssembly 1.0. In *Formal Methods*, Marieke Huisman, Corina Păsăreanu, and Naijun Zhan (Eds.). Vol. 13047. Springer International Publishing, Cham, 61–79. doi:10.1007/978-3-030-90870-6_4

Conrad Watt, Maja Trela, Peter Lammich, and Florian Märkl. 2023. WasmRef-Isabelle: A Verified Monadic Interpreter and Industrial Fuzzing Oracle for WebAssembly. *Proceedings of the ACM on Programming Languages* 7, PLDI (June 2023), 100–123. doi:10.1145/3591224

Yannick Zakowski, Calvin Beck, Irene Yoon, Ilia Zaichuk, Vadim Zaliva, and Steve Zdancewic. 2021. Modular, Compositional, and Executable Formal Semantics for LLVM IR. *Proceedings of the ACM on Programming Languages* 5, ICFP (Aug. 2021), 1–30. doi:10.1145/3473572

C. Zhang, L. Jia, W. Zhang, and N. Wen. 2024. Functional Programming Paradigm of Python for Scientific Computation Pipeline Integration. *arXiv preprint arXiv:2405.16956* (2024). doi:10.48550/ARXIV.2405.16956

Jianzhou Zhao, Santosh Nagarakatte, Milo M.K. Martin, and Steve Zdancewic. 2012. Formalizing the LLVM Intermediate Representation for Verified Program Transformations. *ACM SIGPLAN Notices* 47, 1 (Jan. 2012), 427–440. doi:10.1145/2103621.2103709